

1. What is the main difference between a process and a thread in terms of resource sharing? (2)

A process has its own independent address space and resources (memory, file descriptors, etc.). Threads within the same process share the process's address space and resources (code, data, heap, open files), but each has its own stack, registers, and PC.

2. State two reasons why an operating system provides virtualization of resources. (2)

- o To share a single physical resource among many programs/users (e.g., CPU via time-slicing).
- o To provide isolation/protection and a simpler, uniform abstraction (e.g., virtual memory) so programs don't conflict.

3. Explain the roles of the Program Counter (PC) and the Stack Pointer (SP) registers in the execution of a program. (2)

PC: Holds the address of the next instruction to fetch and execute.

SP: Points to the top of the current stack; tracks function call frames, local variables, and return addresses.

4. Two threads, A and B, share a global variable counter, initialized to 0. Each thread runs counter++ twice. Recall that counter++ breaks down into three steps: LOAD (read counter into a private register), ADD (add 1 to that register), STORE (write the register back to counter). The OS scheduler interleaves the two threads' steps in this exact order. Fill in every blank value in the table. (4)

#	Step	Register	Value
1	A: LOAD counter	A_reg	0
2	A: ADD 1	A_reg	1

3	B: LOAD counter	B_reg	0
4	A: STORE	counter	1
5	B: ADD 1	B_reg	1
6	B: STORE	counter	1
7	A: LOAD counter	A_reg	1
8	B: LOAD counter	B_reg	1
9	A: ADD 1	A_reg	2
10	B: ADD 1	B_reg	2
11	A: STORE	counter	2
12	B: STORE	counter	2

1. Consider the following program: (6)

```
int main() {
    printf("A\n");
    int rc1 = fork();
    if (rc1 == 0) {
        printf("B\n");
        int rc2 = fork();
        if (rc2 == 0) {
            printf("C\n");
        } else {
            printf("D\n");
        }
    } else {
        printf("E\n");
    }
    return 0;
}
```

(a) How many total processes execute this program?

Four

(b) Describe the full process hierarchy – who is the parent of whom? Is there a grandparent/grandchild relationship anywhere?

P0 (main) -> P1 -> P2. (P0 is grandparent of p2)

(c) For each process, list which letter(s) it prints.

P0: A, E

P1: B, D

P2: C

2. A process enters the following loop and never makes a single system call: (4)

```
while (1) {  
    ; // do nothing, forever  
}
```

Explain, step by step, how the OS is nonetheless able to reclaim the CPU from this process.

- o The OS sets a hardware timer to generate periodic interrupts (e.g., every few ms).
 - o When the timer interrupt fires, the CPU switches to the kernel's interrupt handler (Transition from User to kernel mode).
 - o The kernel saves the process's state (PC, registers) and runs the scheduler.
 - o The scheduler may preempt the process and run another one, thus reclaiming the CPU.
-

1. (2 marks) Match the following scheduling algorithms with their characteristics:

- FCFS -> a
- SJF -> d
- STCF -> c
- MLFQ -> b

Options:

- a) First job to arrive is the first to run
- b) Uses multiple priority queues with varying time slices
- c) Uses preemption to always run the job with the least time left
- d) Chooses job with shortest burst time

2. (3 marks) Three processes arrive as follows:

- P1: Arrival = 0, Burst = 6
- P2: Arrival = 1, Burst = 3
- P3: Arrival = 2, Burst = 5

Draw Gantt charts for:

- a) Non-preemptive SJF
- b) Preemptive STCF

Also compute response time and turn around time for each process.

SJF:

Gantt: | P1 | P2 | P3 | (0-6, 6-9, 9-14)

Response time: P1 = 0, P2 = 5, P3 = 7

Turnaround time: P1 = 6, P2 = 8, P3 = 12

STCF:

Gantt: | P1 | P2 | P1 | P3 | (0-1, 1-4, 4-9, 9-14)

Response time: P1 = 0, P2 = 0, P3 = 7

Turnaround time: P1 = 9, P2 = 3, P3 = 12

3. (3 marks) Assume a Multilevel Feedback Queue (MLFQ) with the following configuration:

- Queue 1: Time slice = 2 (highest Priority)
- Queue 2: Time slice = 4
- Queue 3: FCFS

Three jobs arrive at time 0:

- J1: Burst = 4
- J2: Burst = 8
- J3: Burst = 6

Draw the scheduling sequence and show the placement of each job and the transition with time.

```
Gantt:    Q1(2ms) | J1 | J2 | J3 |
           Q2(4ms)                J1 | J2 | J3 |
           Q3                        | J2 |
```

```
Gantt: | J1 | J2 | J3 | J1 | J2 | J3 | J2 |
Times: 0-2, 2-4, 4-6, 6-8, 8-12, 12-16, 16-18.
```

4. (2 marks) Write a short pseudocode for Shortest Job First (SJF) scheduling. Focus on:

- Sorting jobs by burst time
- Executing them in order

SJF(jobs):

```
sort jobs by burst_time ascending
current_time = 0
for each job in sorted order:
    current_time += job.burst
```

```
job.finish = current_time  
job.turnaround = job.finish - job.arrival  
job.response = job.start - job.arrival
```

1. Lottery Scheduling

A Lottery scheduler assigns tickets to three processes as follows:

Process	Tickets
Alpha	5
Beta	3
Gamma	2

The scheduler holds a lottery by generating a single random number between 1 and the total number of tickets. The process that holds the winning ticket gets to run.

a) Calculate the total number of tickets.

$5 + 3 + 2 = 10$ tickets

b) Determine the range of ticket numbers that will lead to each process winning. (Show the winning range for each).

Alpha: 1-5, Beta: 6-8, Gamma: 9-10

c) A random number is drawn: 7. Which process wins the lottery and is scheduled to run?

Beta

2. Explain how MQMS addresses the two problems identified in SQMS?

SQMS problems: Cache affinity, Lock contention (not scalable)

MQMS approach resolves both of these issues. Please refer to chapter 10 for further details.

1. A Stride scheduler uses a large constant value of 10,000 to calculate a process's stride. You are given three processes with the following ticket allocations:

Process	Tickets
---------	---------

P1	10
----	----

P2	20
----	----

P3	30
----	----

The current "pass" values for the processes are:

- P1 Pass = 2,500
- P2 Pass = 1,500
- P3 Pass = 1,000

a) Calculate the stride for each process.

P1: 1000, P2: 500, P3: 333.33 or 333

b) Based on the current pass values, which process (P1, P2, or P3) will be scheduled to run now?

P3(lowest pass value)

c) After this process runs, what will its new pass value be?

1000 + 333 = 1333

1. Consider the following C code: (2)

```
int *p = malloc(sizeof(int));
*p = 42;
printf("%p\n", p);
printf("%p\n", &p);
printf("%d\n", *p);
```

Suppose `p` is a local variable inside `main()`. Which statement correctly describes what `p`, `&p`, and `*p` represent?

- a. `p` is the address of the heap memory, `&p` is the address of the pointer variable (typically on the stack), and `*p` is the value stored in the heap memory.
- b. `p` is the value stored in the heap, `&p` is the address of the heap memory, and `*p` is the address of the pointer variable.
- c. `p` and `&p` both refer to the heap, while `*p` refers to the stack.
- d. `p` is the address of the pointer variable on the stack, `&p` is the value stored in the heap, and `*p` is the address of the heap memory.

Statement a

2. Four CPU-bound processes are running on a single CPU under the Completely Fair Scheduler (CFS). The scheduler uses:
`sched_latency = 24 ms` and `min_granularity = 3 ms`.

Process	Nice Value	Weight
P1	0	1024
P2	5	335
P3	10	110
P4	-5	3121

a. Calculate the actual runtime (time slice) each process receives in one scheduling period. (2)

P1: $(1024/4590) \times 24 \approx 5.36$ ms

P2: $(335/4590) \times 24 \approx 1.75$ ms $\rightarrow$ but `min_granularity` = 3 ms, so 3 ms (clamped up)

P3: $(110/4590) \times 24 \approx 0.58$ ms $\rightarrow$ 3 ms (clamped)

P4: $(3121/4590) \times 24 \approx 16.32$ ms

b. Calculate the vruntime accumulated by each process after one full iteration, assuming each starts with vruntime = 0. (2)

P1: $5.36 \times (1024/1024) = 5.36$

P2: $3 \times (1024/335) \approx 9.17$

P3: $3 \times (1024/110) \approx 27.93$

P4: $16.32 \times (1024/3121) \approx 5.36$

c. After the first iteration, a new process P5 arrives with `nice` = 0 (`weight` = 1024). What would be the vruntime value assigned to this process? (2)

CFS assigns new processes `vruntime` = `min_vruntime` of the runqueue (to prevent starvation). Assuming `min_vruntime` $\approx$ 5.36, P5 gets $\approx$ 5.36 (the current minimum vruntime).

3. What's a dangling pointer. Explain with an example. (2)

A pointer that points to memory that has been freed/deallocated, but still holds the old address. Accessing it causes undefined behavior.

Example:

```
int *p = malloc(sizeof(int));  
  
free(p);  
  
*p = 10;    // dangling pointer - memory already freed
```

1. Consider a system with a 32-bit virtual address space. The top 2 bits of each virtual address are used as the segment selector, and the remaining 30 bits represent the offset.

a. What is the maximum possible size of a segment? (2)

2^{30} bytes = 1 GB

b. How many different segments can be identified using the segment selector? (2)

$2^2 = 4$ segments

c. Assume now that we have 13 bits available to represent the offset. What would be the maximum possible size of a segment? (2)

$2^{13} = 8$ KB max segment size

2. Consider a system with a 14-bit virtual address space. The top 2 bits of each virtual address are used as the segment selector, and the remaining 12 bits represent the offset.

The segments are configured as follows:

Segment	Selector	Virtual Address Range	Physical Base	Size	Growth
Heap	01	4 KB-8 KB	20 KB	3 KB	Up
Stack	11	12 KB-16 KB	28 KB	2 KB	Down

Translate each of the following virtual addresses into a physical addresses.

a. 6,596 (decimal) (2)

Heap segment (virtual address range is 4kb-8kb) and 6596 falls within this range.

Offset = $6596 - 4096 = 2500$

Check bounds: Since $2500 < 3$ KB, it's within bounds.

Physical address = Base + offset

$= 20\text{kb} + 2500 = 22980$

b. 15,260 (decimal) (2)

Stack segment (virtual address range is 12kb-16kb) and 15260 falls within this range.

Offset = 15260 - 12kb = 2972

Since stack grows negatively, subtract it from the max segment size (4096)

Offset = 2972 - 4096 = 1124

Check bounds: Since $|-1124| < 2 \text{ KB}$, it's within stack bounds.

Physical address = Base + offset

= 28kb + (-1124) = 27548

4. Consider a system with a 14-bit virtual address space. The top 2 bits are used as the segment selector, and the remaining 12 bits are used as the offset. A process generates the virtual address 6596.

c. Convert the virtual address 6596 from decimal to its 14-bit binary representation. (2)

01 100111000100

d. Using SEG_MASK and SEG_SHIFT, extract the segment number. (2)

VA: 01 100111000100

SEG_MASK: 11 000000000000

Result: 01 000000000000 >> 01 (since SEG_SHIFT = 12)

Hence the segment number is 01.

e. Explain why the SEG_SHIFT operation is necessary after applying SEG_MASK. (2)

After applying SEG_MASK, the segment bits are still in their original high positions (bit 13-12). Shifting right by 12 moves them to the least significant bits, giving a small integer (0-3) that can be used directly as an index/selector value.